

Phase 1: Onboarding & Profile Setup — Full Baby Steps (with Git)

Step 0: Shared Setup (do this together, once)

1. One person creates a new repo on GitHub called `pulsetag`
 2. Add a `.gitignore` (choose Node + Python templates, or add manually: `venv/`, `node_modules/`, `*.db`, `.env`)
 3. Clone it locally: `git clone https://github.com/<your-username>/pulsetag.git`
 4. Inside the repo, create two folders: `frontend/` and `backend/`
 5. Create a `main` branch (default) — both people will work on their own branch and merge into `main`
 6. Push this initial structure: `git add . → git commit -m "initial repo structure" → git push origin main`
 7. Person1 creates their own branch: `git checkout -b frontend-phase1`
 8. Person2 creates their own branch: `git checkout -b backend-phase1`
-

Person1 (Frontend) — Baby Steps

1. Make sure you're on your branch: `git checkout frontend-phase1`
2. Go into the folder: `cd frontend`
3. Create 3 files: `index.html`, `style.css`, `script.js`
4. In `index.html`, add basic HTML skeleton (`<!DOCTYPE html>`, `<head>`, `<body>`), link `style.css` in `<head>` and `script.js` at the end of `<body>`
5. Save and commit: `git add . → git commit -m "add basic file structure"`
6. Add a `<div id="language-screen">` with a heading "Choose your language" and 8 buttons (Hindi, English, Tamil, Bengali, Marathi, Telugu, Gujarati, Punjabi)
7. Commit: `git add . → git commit -m "add language selection screen"`
8. In `script.js`, add a click listener on each language button — for now just `console.log` the selected language
9. Commit: `git commit -am "add language button logic"`
10. Add a `<div id="onboarding-screen" style="display:none">` with: Aadhaar input, "Send OTP" button, OTP input (hidden), "Verify" button
11. Commit: `git commit -am "add onboarding screen markup"`

12. In `script.js`, write `showOnboarding()` — hides language screen, shows onboarding screen. Call it when any language button is clicked
13. Commit: `git commit -am "wire up language to onboarding transition"`
14. Add fake OTP logic in `script.js`: "Send OTP" reveals the OTP input; "Verify" checks if typed OTP equals `1234` — if yes, show text "ABHA ID created"
15. Commit: `git commit -am "add fake OTP verify flow"`
16. Add a `<div id="profile-form" style="display:none">` with inputs: Name, Blood Group (dropdown), Allergies, Conditions, Medications, and a "Save" button
17. Commit: `git commit -am "add profile form markup"`
18. In `script.js`, show `profile-form` after OTP verification succeeds
19. Commit: `git commit -am "show profile form after otp success"`
20. On "Save" click, collect all form values into one JS object, `console.log` it
21. Commit: `git commit -am "collect profile form data on save"`
22. Push your branch: `git push origin frontend-phase1`

Test: Open `index.html` in a browser → click language → onboarding appears → any Aadhaar → "Send OTP" → type `1234` → "Verify" → success message → profile form appears → fill and "Save" → check console shows the object.

Person2 (Backend) — Baby Steps

1. Make sure you're on your branch: `git checkout backend-phase1`
2. Go into the folder: `cd backend`
3. Create a virtual environment: `python -m venv venv`
4. Activate it: `source venv/bin/activate` (Mac/Linux) or `venv\Scripts\activate` (Windows)
5. Install packages: `pip install fastapi uvicorn sqlalchemy`
6. Save dependencies: `pip freeze > requirements.txt`
7. Commit: `git add . → git commit -m "set up backend environment"`
8. Create `main.py` with a basic FastAPI app and one test route `GET /` returning `{"status": "ok"}`
9. Run it: `uvicorn main:app --reload` — check `http://localhost:8000` shows `{"status": "ok"}`
10. Commit: `git commit -am "add basic fastapi app with health check route"`
11. Create `database.py` — set up SQLite connection string `sqlite:///./pulsetag.db`
12. Commit: `git commit -am "add database connection setup"`

13. Create `models.py` — define `User` table with columns: `id`, `name`, `abha_id`, `blood_group`, `allergies`, `conditions`, `medications`, `phone`
14. Commit: `git commit -am "add user model"`
15. In `main.py`, import models and create tables on app startup
16. Commit: `git commit -am "create tables on startup"`
17. Add route `POST /verify-otp` — takes Aadhaar + OTP, checks OTP equals hardcoded `1234`, returns fake `abha_id` like `ABHA-12345` if correct, else returns an error
18. Commit: `git commit -am "add verify-otp route"`
19. Add route `POST /profile` — takes name, `abha_id`, `blood_group`, `allergies`, `conditions`, `medications`, saves a row, returns new user's `id`
20. Commit: `git commit -am "add profile save route"`
21. Add CORS middleware (allow all origins for now, since it's a hackathon) so the frontend can call these routes
22. Commit: `git commit -am "add cors middleware"`
23. Push your branch: `git push origin backend-phase1`

Test: With the server running, use `curl -X POST http://localhost:8000/verify-otp -H "Content-Type: application/json" -d '{"aadhaar": "1234567890123", "otp": "1234"}'` → confirm you get back an `abha_id`. Then similarly `curl -X POST http://localhost:8000/profile ...` with sample data → confirm a new row exists via `sqlite3 backend/pulsetag.db "SELECT * FROM users;"`.

Integration Step (end of Phase 1 — do this together)

1. Both push your branches (steps above already done)
2. Open a Pull Request from `frontend-phase1` into `main`, review, merge
3. Open a Pull Request from `backend-phase1` into `main`, review, merge
4. Both pull the latest `main`: `git checkout main` → `git pull origin main`
5. Confirm: frontend folder has working forms with console logs; backend folder has working `/verify-otp` and `/profile` routes tested via `curl`
6. Do NOT connect them yet — that's Phase 2. This step is just to confirm both pieces exist cleanly in `main` before building the connection.